Formulación y Búsqueda Ciega en Espacios de Estados

Tarea Individual DG_S4

Jair Lezcano

Escuela Politécnica Nacional

Enlace al repositorio de GitHub: jairdavid06/IA-Trabajos

1. Objetivo general

Modelar matemáticamente el problema clásico de inteligencia artificial de los Misioneros y Caníbales, abstraerlo a un script en Python mediante estructuras de control y estados discretos, e implementar estrategias de búsqueda ciega como BFS y DFS que permitan evaluar y comparar empíricamente su rendimiento computacional, optimalidad y eficiencia.

1.1. Objetivos específicos

- Formalizar teóricamente el problema, detallando con precisión matemática el conjunto de estados, el estado inicial, el estado meta, las acciones permitidas y la función de transición sujeta a restricciones.
- Diseñar la estructura lógica en Python, construyendo funciones basadas en tuplas para representar los estados del sistema y filtrar los movimientos seguros en ambas orillas del río.
- Integrar los algoritmos de búsqueda implementando correctamente una cola (FIFO) para la estrategia BFS y una pila (LIFO) para DFS, asegurando la correcta reconstrucción de la ruta de solución.
- Ejecutar una evaluación midiendo y contrastando métricas clave de rendimiento, como el tiempo de ejecución, el número de nodos expandidos, la cantidad de estados descubiertos y el factor de ramificación efectivo.

2. Introducción

En la disciplina de la Inteligencia Artificial, la formalización y resolución de problemas mediante la exploración de espacios de estados constituyen mecanismos fundamentales para comprender cómo los agentes inteligentes planifican acciones bajo determinadas restricciones lógicas. Uno de los ejemplos clásicos de este enfoque es el problema de los Misioneros y Caníbales, que permite estudiar un entorno acotado pero riguroso para modelar matemáticamente situaciones, definir estados iniciales y metas, y establecer funciones de transición sujetas a restricciones.

El presente trabajo aborda la abstracción y codificación de este problema clásico utilizando Python, estructurando un entorno que separa los componentes del problema y sus restricciones.

Específicamente, se implementan los algoritmos de búsqueda en anchura (BFS) y búsqueda en profundidad (DFS), empleando estructuras de datos fundamentales como colas y pilas, respectivamente, para recorrer el espacio de estados y reconstruir trayectorias de solución válidas.

Para validar el comportamiento de ambas estrategias, se lleva a cabo un *benchmark* que permite contrastar métricas como el tiempo de ejecución, el número de nodos expandidos, la cantidad de estados descubiertos y el factor de ramificación efectivo (b^*). Con base en estos resultados, el informe analiza por qué BFS garantiza encontrar una solución óptima en cantidad de movimientos cuando todos los movimientos tienen el mismo costo, mientras que DFS puede encontrar una solución válida sin garantizar que sea la más corta.

3. Formulación del problema

3.1. Descripción del problema

El dilema de los Misioneros y Caníbales representa un problema de espacio de estados dentro de la Inteligencia Artificial, encuadrado en la categoría de problemas de planificación y búsqueda ciega. La situación plantea un escenario en el cual un grupo compuesto por tres misioneros y tres caníbales se encuentra en la orilla izquierda de un río y necesita trasladarse hacia la orilla derecha. Para completar este cruce, dispone de una única embarcación con capacidad para transportar un máximo de dos personas y un mínimo de una, ya que el bote no puede desplazarse de manera autónoma.

La dificultad central del problema reside en una restricción: en ninguna de las dos orillas la cantidad de caníbales puede superar a la de misioneros cuando haya al menos un misionero presente. Si en algún instante los caníbales superan numéricamente a los misioneros en una orilla donde hay misioneros, el estado se considera inválido.

Bajo este contexto, el desafío consiste en determinar una secuencia ordenada y finita de movimientos seguros de ida y vuelta que permita transportar exitosamente a las seis personas al otro lado del río. El problema exige coordinar los viajes de la embarcación para evitar estados peligrosos, ciclos innecesarios o movimientos que infrinjan las reglas. Por estas características, constituye un modelo apropiado para probar y contrastar la eficiencia temporal, espacial y la capacidad de reconstrucción de rutas de los algoritmos de búsqueda ciega.

3.2. Conjunto de estados

El conjunto de estados, denotado formalmente como S, representa el universo de configuraciones que puede adoptar el sistema durante el proceso de planificación y cruce del río. Para modelar este dominio de manera eficiente y compacta, cada estado se define mediante una tupla tridimensional:

$$s = (m_{izq}, c_{izq}, b)$$

Cada elemento captura una variable clave del entorno:

1. **Número de misioneros en la orilla izquierda (m_{izq}):** representa la cantidad de misioneros presentes en la orilla de origen. Este valor puede tomar cualquier número entero entre 0 y 3.

2. **Número de caníbales en la orilla izquierda (c_{izq}):** representa la cantidad de caníbales presentes en la orilla de origen. Este valor también pertenece al conjunto de enteros $\{0, 1, 2, 3\}$.
3. **Posición del bote (b):** es una variable binaria que indica la ubicación de la embarcación. Toma el valor de 1 si el bote se encuentra en la orilla izquierda y 0 si se encuentra en la orilla derecha.

No es necesario incluir explícitamente las poblaciones de la orilla derecha dentro de la tupla, ya que estas se deducen mediante el principio de conservación de la población total. Por tanto, los misioneros de la derecha equivalen a $3 - m_{izq}$ y los caníbales de la derecha a $3 - c_{izq}$.

3.2.1. Dimensión teórica del espacio de estados

Desde una perspectiva combinatoria, el espacio matemático total se calcula multiplicando las posibles variaciones de sus tres componentes:

$$4 \text{ (misioneros)} \times 4 \text{ (caníbales)} \times 2 \text{ (bote)} = 32$$

Por tanto, existen 32 combinaciones matemáticas posibles. Sin embargo, no todas forman parte del espacio de estados válidos del problema. Algunas configuraciones infringen las restricciones de seguridad o no pueden alcanzarse mediante las acciones permitidas. En consecuencia, los algoritmos operan sobre un subconjunto de estados válidos y alcanzables.

3.3. Estado inicial y estado meta

En la teoría de búsqueda y planificación por espacios de estados, la definición del punto de partida y de la meta permite establecer los límites del problema y determinar cuándo un algoritmo ha completado su tarea. Para el caso de los Misioneros y Caníbales, estos puntos se definen de la siguiente manera:

- **Estado inicial (S_0):** se define mediante la tupla $(3, 3, 1)$. Esta configuración representa el punto de partida, donde los tres misioneros y los tres caníbales se encuentran en la orilla izquierda y la embarcación también está en esa orilla. Como consecuencia del modelo de conservación, la orilla derecha se encuentra inicialmente vacía.
- **Estado meta (S_{meta}):** se define mediante la tupla $(0, 0, 0)$. Al alcanzar este estado, los tres misioneros y los tres caníbales han cruzado a la orilla derecha y el bote se encuentra también en la orilla de destino. Una secuencia de acciones legales que transforme el estado inicial en este estado objetivo constituye una solución válida.

3.4. Acciones permitidas

El conjunto de acciones u operadores legales, denotado como A, define los movimientos de transporte que pueden realizarse en cada paso. Debido a la capacidad de la embarcación, cada viaje debe transportar al menos una persona y como máximo dos. Las acciones posibles se representan mediante tuplas de la forma (m_{viaje}, c_{viaje}), donde cada valor indica la cantidad de misioneros y caníbales que suben al bote:

- **Acción** $(1, 0)$**:** transportar un misionero.
- **Acción** $(2, 0)$**:** transportar dos misioneros.
- **Acción** $(0, 1)$**:** transportar un caníbal.

- **Acción** $(0, 2)$**:** transportar dos caníbales.
- **Acción** $(1, 1)$**:** transportar un misionero y un caníbal conjuntamente.

Cualquier combinación que supere los dos pasajeros, intente mover cero personas o pretenda trasladar más individuos de los que están presentes en la orilla de partida queda descartada del conjunto de acciones válidas.

3.5. Función de transición

La función de transición, denotada como $T(s, a)$, define cómo evoluciona el sistema al aplicar una acción válida. Toma como entradas un estado actual $s = (m_{izq}, c_{izq}, b)$ y una acción legal $a = (m_{viaje}, c_{viaje})$, y genera un nuevo estado sucesor $s' = (m'_{izq}, c'_{izq}, b')$.

El comportamiento depende de la posición actual del bote. Si el bote se encuentra en la orilla izquierda ($b = 1$), la embarcación parte hacia la derecha. Los pasajeros abandonan la orilla de origen, por lo que se restan las cantidades correspondientes y el bote cambia su posición a 0:

$$s' = (m_{izq} - m_{viaje},\ c_{izq} - c_{viaje},\ 0)$$

Si el bote se encuentra en la orilla derecha ($b = 0$), realiza el viaje de retorno hacia la izquierda. Los pasajeros se reincorporan a la orilla de origen, por lo que se suman las cantidades correspondientes y el bote cambia su posición a 1:

$$s' = (m_{izq} + m_{viaje},\ c_{izq} + c_{viaje},\ 1)$$

3.6. Restricciones

Para garantizar la validez del estado generado tras cada transición, el sistema implementa filtros lógicos que impiden explorar configuraciones inviables. En primer lugar, las cantidades de misioneros y caníbales en la orilla izquierda deben mantenerse dentro del intervalo de cero a tres, y la posición de la embarcación debe ser estrictamente binaria.

La restricción principal establece que, en cada orilla, los caníbales no pueden superar a los misioneros cuando haya al menos un misionero presente. Esta condición debe cumplirse simultáneamente en ambas orillas. Por tanto, si una acción genera un estado que infringe el equilibrio en cualquiera de ellas, dicho estado se descarta inmediatamente. De esta manera, el algoritmo explora únicamente configuraciones que respetan las condiciones de seguridad del problema.

3.7. Criterios de solución

Para que una secuencia de pasos sea considerada una solución válida, debe constituir una trayectoria finita y ordenada de acciones legales, $P = (a_1, a_2, ..., a_k)$, que aplique sucesivamente la función de transición sobre el estado inicial hasta alcanzar el objetivo. Además, todos los estados intermedios deben pertenecer al conjunto de estados válidos ($s_i \in S_{vlidos}$):

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \cdots \xrightarrow{a_k} s_k = S_{meta}$$

La evaluación también contempla la eficiencia en el uso de recursos informáticos. Para ello, se consideran métricas como la cantidad de nodos expandidos, la cantidad de estados descubiertos, el tiempo de ejecución y la profundidad de la ruta encontrada. De esta manera, se puede analizar tanto la validez de la solución como el comportamiento operativo de los algoritmos implementados.

4. Implementación en Python

4.1. Representación del estado y movimientos permitidos del bote

La implementación comienza con la modelación de la información y la dinámica del problema. El sistema define el estado inicial mediante la tupla `ESTADO_INICIAL = (3, 3, 1)` y el estado meta como `ESTADO_OBJETIVO = (0, 0, 0)`. Cada configuración almacena de forma compacta las poblaciones en la orilla de origen y la posición binaria de la embarcación.

Para los desplazamientos, el código establece el conjunto `MOVIMIENTOS_PERMITIDOS_BOTE`, que contiene las combinaciones autorizadas de pasajeros: el traslado individual de un misionero o un caníbal, el traslado de dos misioneros o dos caníbales, y el traslado conjunto de un misionero y un caníbal.

4.2. Módulo de validación de estados

El módulo de validación actúa como un filtro que comprueba la viabilidad física y lógica de cada configuración candidata antes de incorporarla al proceso de búsqueda. La función `es_orilla_segura` evalúa individualmente cada orilla: si no hay misioneros, la orilla se considera segura; si hay misioneros, su cantidad debe ser mayor o igual que la de caníbales.

Por otro lado, la función `es_estado_valido` procesa el estado global mediante las variables `misioneros_izq`, `canibales_izq` y `posicion_bote`. Comprueba que las cantidades se mantengan entre cero y tres y que la posición del bote sea binaria. Además, deduce la distribución de personas en la orilla opuesta y aplica la condición de seguridad a ambas orillas simultáneamente. De este modo, descarta las configuraciones que infringen las restricciones.

4.3. Módulo de sucesores y función de transición

La función `obtener_sucesores` modela los desplazamientos posibles a partir del estado actual y del conjunto de movimientos permitidos. Según la ubicación del bote, comprueba la disponibilidad de pasajeros en la orilla de partida, actualiza las cantidades de misioneros y caníbales y alterna la posición de la embarcación. Finalmente, devuelve únicamente los estados sucesores que superan los filtros de validez.

4.4. Búsqueda en anchura (BFS)

La búsqueda en anchura (BFS) explora el espacio de estados por niveles. Primero examina los estados situados a una misma profundidad y después avanza al siguiente nivel. En la implementación, utiliza una estructura FIFO (*First-In, First-Out*) mediante una cola optimizada con `deque([inicial])`. También emplea un conjunto `visitados` para evitar explorar repetidamente un mismo estado y un diccionario `padre` para registrar los antecesores y reconstruir la ruta.

En cada iteración, el algoritmo extrae el primer elemento de la cola mediante `popleft()` y comprueba si corresponde al estado objetivo. Si no es así, obtiene sus sucesores con

`obtener_sucesores(estado)`. Cada sucesor que todavía no haya sido visitado se registra, se incorpora al final de la cola y se vincula con su estado antecesor. Como todos los movimientos del problema tienen el mismo costo, BFS garantiza que la primera solución encontrada contiene el menor número de cruces posible.

4.5. Búsqueda en profundidad (DFS)

La búsqueda en profundidad (DFS) adopta una estrategia de exploración vertical: prioriza avanzar por una rama hasta alcanzar el objetivo o agotar las alternativas disponibles. En la implementación, este comportamiento se gestiona mediante una pila LIFO (*Last-In, First-Out*), representada por `pila = [inicio]`. Al igual que BFS, utiliza un conjunto `visitados` para evitar repetir estados y un diccionario `padre` para reconstruir la ruta.

Operativamente, el algoritmo extrae el elemento situado en el tope de la pila mediante `pop()`, comprueba si representa la meta y, si no es así, obtiene sus sucesores y los incorpora a la pila. Cuando una rama deja de ofrecer estados nuevos, el algoritmo puede continuar con las alternativas que permanecen pendientes en la estructura. Aunque DFS puede encontrar rápidamente una solución, no garantiza que esta tenga el menor número de movimientos.

4.6. Métricas y rendimiento

El módulo de métricas registra el comportamiento de los algoritmos para permitir una comparación cuantitativa. Durante la ejecución, contabiliza los nodos expandidos, obtiene la cantidad de estados registrados en el conjunto `visitados` y mide el tiempo transcurrido mediante `time.perf_counter()`.

Asimismo, la función auxiliar `calcular_factor_ramificacion_efectivo` utiliza la profundidad de la solución y el número de estados descubiertos para estimar el factor de ramificación efectivo (b^*), mediante un procedimiento de búsqueda binaria. Finalmente, la función `ejecutar_benchmark` reúne los indicadores y los presenta en una tabla comparativa. Cabe aclarar que la cantidad de estados visitados funciona como una medida del volumen de estados registrados, pero no constituye una medición directa del consumo de memoria RAM.

5. Resultados experimentales

Los resultados del *benchmark* permiten observar las diferencias entre BFS y DFS en la forma de recorrer los estados y en los recursos utilizados para encontrar una solución. Ambos algoritmos alcanzaron el estado objetivo en 11 pasos, aunque siguieron recorridos distintos y presentaron diferencias en la cantidad de nodos expandidos y en el tiempo registrado.

La Tabla 1 resume las métricas obtenidas durante la ejecución:

Tabla 1: Resumen de métricas de rendimiento y eficiencia operativa entre BFS y DFS.

Métrica evaluada	BFS (anchura)	DFS (profundidad)
Solución encontrada	Sí (11 pasos)	Sí (11 pasos)
Nodos expandidos	14	11
Estados descubiertos (visitados)	15	15
Tiempo de ejecución	0.0000433 s	0.0000301 s

Métrica evaluada	BFS (anchura)	DFS (profundidad)
Factor de ramificación efectivo (b^*)	1.0397	1.0397

En cuanto al número de nodos expandidos, BFS registró 14, mientras que DFS necesitó expandir 11. Esta diferencia se explica por sus estrategias de exploración. BFS revisa los estados por niveles y puede expandir alternativas que no forman parte de la ruta final, pero que deben considerarse antes de avanzar a una profundidad mayor. DFS, en cambio, continúa por una misma rama y, en esta ejecución, alcanzó el objetivo después de expandir menos nodos. Esto también se refleja en el tiempo medido: BFS tardó 0.0000433 segundos y DFS, 0.0000301 segundos. Por tanto, DFS fue ligeramente más rápido en esta prueba. Como los tiempos son muy pequeños, deben interpretarse como resultados de esta ejecución concreta y pueden variar según las condiciones del equipo y la carga del sistema.

En cuanto a las complejidades asintóticas, BFS presenta un costo temporal y espacial exponencial respecto a la profundidad de la solución. En el análisis tradicional se expresa como $O(b^{d+1})$ para tiempo y espacio, donde b representa el factor de ramificación y d la profundidad de la solución. Algunas referencias utilizan la forma simplificada $O(b^d)$. DFS presenta un costo temporal de $O(b^m)$ y un costo espacial de $O(bm)$ en la búsqueda tradicional en árboles, donde m representa la profundidad máxima. Estas expresiones describen el comportamiento teórico y no deben confundirse con las mediciones de una ejecución específica.

En esta implementación hay una particularidad importante: tanto BFS como DFS utilizan un conjunto de estados visitados y un diccionario de padres. Por ello, aunque la frontera de exploración de DFS suele ser menor, ambos algoritmos conservan información de los estados descubiertos. En el *benchmark*, los dos registraron 15 estados visitados. Este dato indica que la cantidad de estados registrados fue la misma, pero no demuestra que el consumo real de memoria RAM también haya sido idéntico. Para comparar la memoria física utilizada sería necesario medir directamente el consumo máximo durante cada ejecución.

El factor de ramificación efectivo también fue igual para ambos algoritmos: 1.0397. Este valor resume el crecimiento efectivo del espacio explorado a partir de la cantidad de estados descubiertos y la profundidad de la solución. En este caso, el valor relativamente cercano a uno es coherente con un problema donde las restricciones de seguridad y la capacidad limitada del bote reducen las transiciones válidas disponibles en cada estado.

La Tabla 2 permite comparar las rutas encontradas. Se observa que ambas parten del mismo estado inicial y alcanzan el mismo estado objetivo, aunque presentan diferencias en los estados intermedios, particularmente en los pasos 1 y 10.

Tabla 2: Comparación de las secuencias de estados obtenidas mediante BFS y DFS.

Paso	Camino BFS	Camino DFS
0	(3, 3, 1)	(3, 3, 1)
1	(3, 1, 0)	(2, 2, 0)
2	(3, 2, 1)	(3, 2, 1)
3	(3, 0, 0)	(3, 0, 0)
4	(3, 1, 1)	(3, 1, 1)
5	(1, 1, 0)	(1, 1, 0)

Paso	Camino BFS	Camino DFS
6	(2, 2, 1)	(2, 2, 1)
7	(0, 2, 0)	(0, 2, 0)
8	(0, 3, 1)	(0, 3, 1)
9	(0, 1, 0)	(0, 1, 0)
10	(1, 1, 1)	(0, 2, 1)
11	(0, 0, 0)	(0, 0, 0)

Es importante analizar si DFS garantizó encontrar el camino más corto. La respuesta es no: al explorar una rama hasta el fondo, puede encontrar una solución a mayor profundidad aunque exista otra más corta. En este caso, ambos algoritmos encontraron una ruta de 11 movimientos, pero esto no significa que DFS garantice ese resultado en cualquier situación. BFS sí garantiza el camino más corto cuando todos los movimientos tienen el mismo costo, porque explora primero los estados de menor profundidad. Por ello, al encontrar la meta en el nivel 11, permite afirmar que no existe una solución con menos movimientos.

6. Conclusiones

La formulación matemática del problema de los Misioneros y Caníbales permitió representar de manera estructurada un entorno de búsqueda mediante estados discretos, acciones permitidas y restricciones de seguridad. La utilización de tuplas para identificar la cantidad de misioneros, caníbales y la posición del bote facilitó la construcción del espacio de estados, mientras que las funciones de validación y transición garantizaron que los movimientos generados respetaran las condiciones establecidas. De esta manera, se consiguió transformar un problema lógico en un modelo computacional que puede ser explorado sistemáticamente mediante algoritmos de búsqueda ciega.

La implementación de BFS y DFS en Python permitió comprobar cómo dos estrategias diferentes pueden resolver un mismo problema utilizando mecanismos de exploración distintos. Ambos algoritmos encontraron una solución válida de 11 movimientos, aunque presentaron diferencias en algunos estados intermedios de sus recorridos. En este caso, BFS permitió confirmar que 11 movimientos corresponden a la solución más corta, mientras que DFS también alcanzó esa misma profundidad sin contar con una garantía general de optimalidad. Esto demuestra que encontrar una solución óptima mediante DFS puede ocurrir dependiendo del orden de exploración, pero no constituye una propiedad garantizada del algoritmo.

Los resultados experimentales evidenciaron que DFS presentó un menor número de nodos expandidos, con 11 frente a los 14 registrados por BFS, además de un menor tiempo de ejecución en la prueba realizada. Este comportamiento se relaciona con su estrategia de profundizar por una rama y alcanzar directamente el objetivo sin explorar tantas alternativas. Por el contrario, BFS necesitó examinar más estados debido a su recorrido por niveles, pero este comportamiento le permitió garantizar la obtención del camino más corto. Asimismo, ambos algoritmos registraron 15 estados descubiertos y un factor de ramificación efectivo de 1.0397, lo que refleja que las restricciones del problema redujeron las posibilidades de expansión y mantuvieron relativamente pequeño el espacio de búsqueda.

Finalmente, se determinó que la eficiencia de BFS y DFS depende de las características del problema y de los objetivos que se persigan durante la búsqueda. BFS resulta especialmente apropiado

cuando es indispensable garantizar una solución con el menor número de movimientos, mientras que DFS puede ser conveniente cuando se busca explorar una alternativa hasta encontrar una solución válida sin priorizar necesariamente la optimalidad. Aunque en este ejercicio DFS obtuvo mejores resultados en cantidad de nodos expandidos y tiempo de ejecución, el tamaño reducido del espacio de estados no permite generalizar esa ventaja a problemas de mayor dimensión. Por tanto, la selección de una estrategia de búsqueda debe considerar conjuntamente la complejidad computacional, el consumo de recursos y las garantías que ofrece cada algoritmo.